

DATA STRUCTURES & ALGORITHMS

Queue Algorithm Design & Analysis with Java

CPU Process Scheduling: FCFS vs Round Robin (TQ = 3)

จัดทำโดย กลุ่มที่ 3
ชื่อ-นามสกุล นธิศ สุขอยู่ รหัสนักศึกษา 67122250033

วิชา Data Structures and Algorithms



วัตถุประสงค์

จากโครงสร้างข้อมูล → สุนโยบายจัดสรรเวลา CPU ที่วัดผลได้

01 ออกแบบ Queue

จัดสรร CPU Time ให้ Process อย่างเป็นระบบ

02 เปรียบเทียบ

FCFS กับ Round Robin ที่ $TQ = 3$

03 วิเคราะห์

Time / Space Complexity และ trade-off

04 ทดลองจริง

รัน Java และวัดประสิทธิภาพ

Input ที่ชัดเจน ทำให้ผลลัพธ์ตรวจสอบได้

ทุก Process เข้าพร้อมกันที่ $t = 0$ • Time Quantum = 3

PROCESS	ARRIVAL	BURST	REMAINING
P1	0	8	8
P2	0	4	4
P3	0	9	9
P4	0	5	5

TARGET OUTPUT

Gantt Chart Log
Waiting Time
Turnaround Time
Context Switches

READY QUEUE
`java.util.ArrayDeque<Process>`

Algorithm A – FCFS

FIFO • มาก่อน รันก่อน • ทำงานจนเสร็จ

```
for each process by arrival:  
    queue.add(process)
```

```
while (!queue.isEmpty()):  
    p = queue.remove()  
    time += p.burstTime
```

จุดเด่น
เขียนง่าย
Overhead ต่ำ

จุดด้อย
Convoy Effect
งานยาวบล็อกงานสั้น

P1 → P2 → P3 → P4

Algorithm B – Round Robin

TIME QUANTUM = 3

```
p = queue.remove()
exec = min[p.remaining, 3]
p.remaining -= exec
```

```
if (p.remaining > 0):
    queue.add(p)
```

CPU หมุนเวียนให้ทุก Process
ครั้งละไม่เกิน 3 หน่วยเวลา

ตอบสนองไว • fairness สูง

แลกกับ overhead จาก context switch

P1 P2 P3 P4 ↻

Queue Trace – 10 Operations

Round Robin • TQ = 3

#	OP	QUEUE ก่อน	QUEUE หลัง	ACTION
1	DEQ	P1 P2 P3 P4	P2 P3 P4	P1 run 3 → rem 5
2	ENQ	P2 P3 P4	P2 P3 P4 P1	P1 → tail
3	DEQ	P2 P3 P4 P1	P3 P4 P1	P2 run 3 → rem 1
4	ENQ	P3 P4 P1	P3 P4 P1 P2	P2 → tail
5	DEQ	P3 P4 P1 P2	P4 P1 P2	P3 run 3 → rem 6
6	ENQ	P4 P1 P2	P4 P1 P2 P3	P3 → tail
7	DEQ	P4 P1 P2 P3	P1 P2 P3	P4 run 3 → rem 2
8	ENQ	P1 P2 P3	P1 P2 P3 P4	P4 → tail
9	DEQ	P1 P2 P3 P4	P2 P3 P4	P1 run 3 → rem 2
10	ENQ	P2 P3 P4	P2 P3 P4 P1	P1 → tail

Correctness = Invariant ที่ยังจริงในทุก iteration

FCFS INVARIANT

หน้าคิวคือ Process ที่มาถึงก่อน
และจะรันจนเสร็จโดยไม่ถูกแทรก

ROUND ROBIN INVARIANT

หน้าคิวรอนานที่สุดจากรอบล่าสุด
รัน $\leq TQ$ แล้วกลับทำคิวหากยังไม่เสร็จ

FIFO preserved $\rightarrow$ no process is lost $\rightarrow$ termination when remaining = 0

Complexity Analysis

ARRAYDEQUE

enqueue / dequeue / peek
 $O(1)$

search / display
 $O(n)$

FCFS

TIME
 $O(n)$

SPACE
 $O(n)$

ROUND ROBIN

TIME
 $O(n \times \max(\text{Burst}) / TQ)$

SPACE
 $O(n)$

Java Implementation Highlights

```
class Process {  
    String id;  
    int arrival, burst, remaining;  
  
    Process copy() {  
        return new Process(id, arrival, burst);  
    }  
}  
  
Deque<Process> ready = new ArrayDeque<>();
```

WHY ARRAYDEQUE?

- cache locality ดีกว่า LinkedList
- ไม่มี node overhead
- addLast / removeFirst เป็น $O(1)$
- รองรับ Java 8+ ด้วย Arrays.asList()

copy() แยก state ของ Process
เพื่อให้ FCFS และ RR ทดสอบจากข้อมูลชุดเดียวกัน

Test Coverage – 6 กรณี

01 NORMAL

P1(8), P2(4), P3(9), P4(5)

02 EMPTY

คิวว่าง []

03 SINGLE

P1(5) • FCFS = RR

04 LARGE

n = 10,000 • memory

05 EQUAL BURST

3, 3, 3 • minimal switch

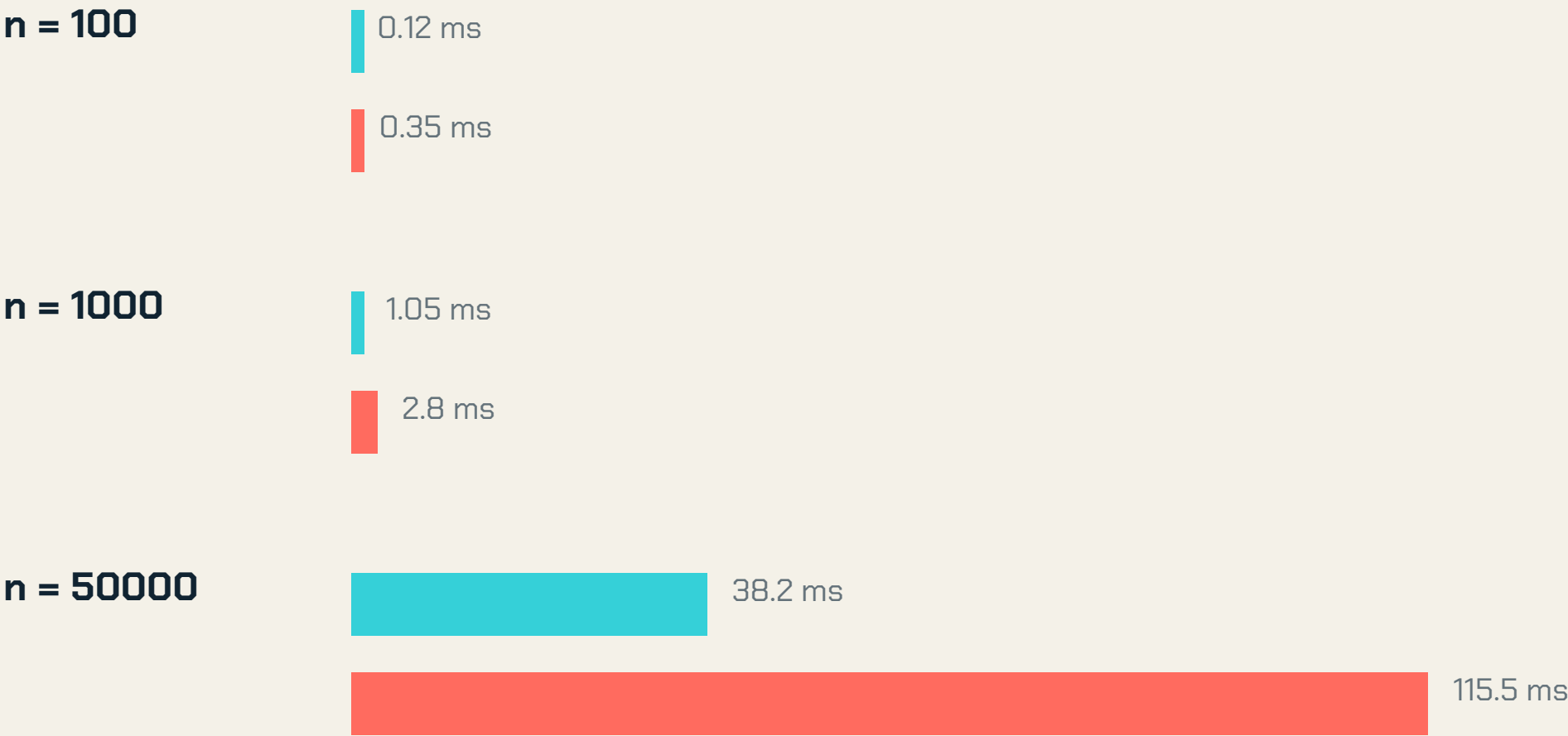
06 CANCEL

ลบกลางคิว / ไม่พบ → false

Experimental Results

Execution Time เฉลี่ย 5 รอบ (ms)

■ FCFS ■ RR



FCFS เร็วกว่า เพราะไม่มี overhead จาก context switching และ re-enqueue

เลือก Algorithm ให้ตรงกับ workload

FCFS

Fairness ต่ำ
Response ช้า
Context switch ต่ำ
เหมาะกับ Batch Processing

ROUND ROBIN

Fairness สูง
Response ดีเยี่ยม
Context switch สูง
เหมาะกับ Interactive System

SIMPLE ≠ FAIR • FAIR ≠ FREE